UNIVERSITY OF DHAKA
Department of Computer Science and Engineering

CSE 2206 — Microcontroller and Embedded System

Lab Assignment 3

BME280 Environmental Sensor Interface

Part A: SPI Communication — Part B: I^2C Communication

Bare-Metal (Register-Level) & HAL / STM32CubeIDE

Course Details

Field	Details
Course Code	CSE 2206
Course Title	Microcontroller and Embedded System
Assignment	Assignment 3 of 4
Part A Topic	SPI Bus — BME280 (4-wire SPI, STM32 as master)
Part B Topic	I^2C Bus — BME280 (2-wire I^2C, STM32 as master)
Platform	STM32F446RE Nucleo-64
IDE	STM32CubeIDE (Bare-Metal & HAL — both required)
Terminal	GTKterm (Linux) / PuTTY (Windows) @ 115200 baud
Marks (Part A)	50
Marks (Part B)	50
Total Marks	**100**

Outcome-Based Education (OBE) Mapping

This assignment is designed in alignment with the OBE framework for CSE 2206. All tasks across both parts map to Course Learning Outcomes (CLOs) and Programme Outcomes (POs).

Course Learning Outcomes (CLOs)

CLO #	Statement	Bloom's Level	Mapped PO
CLO 1	Configure SPI peripheral registers (bare-metal) on STM32 and interface BME280 as SPI slave.	Apply (L3)	PO1, PO5
CLO 2	Configure I²C peripheral registers (bare-metal) on STM32 and interface BME280 as I²C slave.	Apply (L3)	PO1, PO5
CLO 3	Read BME280 calibration data and apply manufacturer compensation formulas for both protocols.	Analyze (L4)	PO2, PO5
CLO 4	Implement identical sensor interfaces using STM32 HAL / CubeIDE and compare both approaches.	Evaluate (L5)	PO3, PO5
CLO 5	Use GTKterm to verify UART output and perform systematic hardware-level debugging.	Apply (L3)	PO4, PO5
CLO 6	Write modular, well-commented embedded C code adhering to professional coding standards.	Create (L6)	PO6

Task-to-CLO Mapping

Task	1	2	3	4	5	6
Part A — Clock, GPIO, UART, TIM6	✓				✓	✓
Part A — SPI2 Configuration (BM)	✓					✓
Part A — BME280 Init & Calibration			✓			✓
Part A — Data Read & Compensation			✓			✓
Part A — HAL / CubeIDE SPI				✓		✓
Part A — Verification Tests	✓		✓	✓	✓	
Part B — I²C1 Configuration (BM)		✓				✓
Part B — BME280 via I²C Init			✓			✓
Part B — Data Read & Compensation			✓			✓
Part B — HAL / CubeIDE I²C				✓		✓
Part B — Verification Tests		✓	✓	✓	✓	
GTKterm Setup (both parts)					✓	
Protocol Comparison (end of document)	✓	✓		✓		✓

1. Real-World Scenario

Embedded systems are at the heart of modern environmental monitoring — from rooftop weather stations to HVAC controllers in smart buildings. Microcontrollers regularly communicate with sensors over serial buses to collect temperature, pressure, and humidity data in real time.

In this assignment you interface the STM32F446RE with the BME280 environmental sensor using *two* different communication protocols: SPI (Part A) and I²C (Part B). Both parts produce identical sensor output; the difference lies entirely in the bus protocol and its register-level configuration.

> **Real-World Context:** Industrial IoT gateways, portable weather instruments, and automotive climate sensors all use the BME280 or similar devices — some over SPI (for speed), others over I²C (to minimise pin count). A practising engineer must be comfortable with both.

2. Objectives

By the end of this assignment, students will be able to:

- Explain SPI and I²C protocols, their differences, and appropriate use cases.
- Configure SPI2 peripheral registers on STM32 at the register level (no HAL).
- Configure I²C1 peripheral registers on STM32 at the register level (no HAL).
- Interface the BME280 sensor as both an SPI slave (Part A) and an I²C slave (Part B).
- Read BME280 calibration parameters and apply the manufacturer compensation formulas.
- Perform unit conversions: Celsius to Fahrenheit, Pascal to hPa.
- Transmit formatted sensor readings to a PC terminal using USART2.
- Use TIM6 hardware interrupts to trigger periodic 1-second data transmissions.
- Install and configure GTKterm on Linux for UART verification.
- Implement both SPI and I²C designs using STM32 HAL in CubeIDE.
- Compare SPI and I²C in terms of pin usage, speed, and implementation complexity.

3. GTKterm — Linux Serial Terminal Setup

GTKterm is required for UART verification in *both* Part A and Part B. Set it up once before starting either part.

3.1 Installation

```
sudo apt update
sudo apt install gtkterm -y
```

```
# Fedora/RHEL: sudo dnf install gtkterm
```

3.2 Identifying the Serial Port

```
ls /dev/ttyACM* /dev/ttyUSB*
dmesg | tail -20 | grep tty
```

The Nucleo's ST-Link typically appears as `/dev/ttyACM0`. A discrete USB-UART adapter (CP2102 / CH340) appears as `/dev/ttyUSB0`.

3.3 Port Access (One-Time Setup)

```
sudo usermod -aG dialout $USER
# Log out and back in for the change to take effect
```

> Without this step, GTKterm will show *"Permission denied"* when opening the port.

3.4 GTKterm Configuration

Launch GTKterm (`gtkterm`), then navigate to **Configuration → Port** and set:

Parameter	Value	Notes
Port	`/dev/ttyACM0`	Or `/dev/ttyUSB0` — see Section 3.2
Baud Rate	115200	Must match USART2 firmware setting
Bits	8	Data bits
Parity	None	
Stop Bits	1	
Flow Control	None	Hardware flow control is not used

Go to **Configuration → Save Configuration** to persist as default.

> **GTKterm Tips:** Press `Ctrl+L` to clear the screen. Use **File → Log to file** to save session output for your report.

4. BME280 Sensor — Common Background

Both Part A and Part B use the same BME280 sensor. This section covers the sensor data, calibration, and compensation formulas common to both protocols.

4.1 Measurements

Measurement	Raw Format	Registers	Physical Range
Temperature	20-bit unsigned	0xFA–0xFC	−40 °C to +85 °C
Pressure	20-bit unsigned	0xF7–0xF9	300 hPa to 1100 hPa
Humidity	16-bit unsigned	0xFD–0xFE	0 % to 100 % RH

4.2 Calibration Parameters

Group	Registers	Used For
`dig_T1, dig_T2, dig_T3`	0x88–0x8D	Temperature compensation
`dig_P1` – `dig_P9`	0x8E–0x9F	Pressure compensation
`dig_H1` – `dig_H6`	0xA1, 0xE1–0xE7	Humidity compensation

4.3 Operating Mode — Indoor Navigation

Setting	Value	Register
Sensor Mode	Normal (continuous)	0xF4 — `mode[1:0] = 11`
Temperature Oversampling	×2	0xF4 — `osrs_t[2:0] = 010`
Pressure Oversampling	×16	0xF4 — `osrs_p[2:0] = 101`
Humidity Oversampling	×1	0xF2 — `osrs_h[2:0] = 001`
IIR Filter Coefficient	16	0xF5 — `filter[2:0] = 100`
Standby Time	0.5 ms	0xF5 — `t_sb[2:0] = 000`

4.4 Unit Conversions

These conversions are identical for both parts:

$$T\,(^\circ\mathrm{F}) = T\,(^\circ\mathrm{C}) \times \frac{9.0}{5.0} + 32.0$$

$$P\,(\mathrm{hPa}) = \frac{P\,(\mathrm{Pa})}{100.0}$$

Humidity is already expressed in % RH — no conversion is needed.

PART A
SPI Communication with BME280

4-Wire SPI — STM32 as Master — Bare-Metal + HAL

A1. SPI Protocol Overview

Serial Peripheral Interface (SPI) is a synchronous, full-duplex, 4-wire serial bus. The master (STM32) drives the clock and initiates all transfers; the slave (BME280) responds when selected via an active-low Chip Select line.

Signal	Full Name	Direction	Purpose
MOSI	Master Out Slave In	Master → Slave	STM32 sends register addresses and write data
MISO	Master In Slave Out	Slave → Master	BME280 returns read data
SCK	Serial Clock	Master → Slave	Clock generated by STM32; synchronises bits
CS	Chip Select (active LOW)	Master → Slave	STM32 pulls LOW to select BME280

A1.1 Clock Mode

Parameter	Value	Meaning
CPOL	0	SCK idle LOW
CPHA	0	Data sampled on first (rising) edge
SPI Mode	Mode 00	BME280 default; lowest power

A1.2 Read / Write Encoding

- **Write:** send `(reg_addr & 0x7F)` — MSB = 0
- **Read:** send `(reg_addr | 0x80)` — MSB = 1; address auto-increments for burst reads

A2. Hardware Connections — SPI

SPI Signal	STM32 Pin	Alt Function	BME280 Pin
MOSI (Data Out)	PC1	AF7	SDI
MISO (Data In)	PC2	AF5	SDO
SCK (Clock)	PC7	AF5	SCK
CS (Chip Select)	PB9 (GPIO Output)	—	CSB
UART TX	PA2	AF7 (USART2)	—
UART RX	PA3	AF7 (USART2)	—

A3. Bare-Metal Implementation (Register-Level)

Follow all steps in order. Do **NOT** use any HAL, SPL, or middleware functions in this section.

A3.1 Step A3.1 — Clock Configuration (180 MHz)

- Enable HSE: `RCC_CR` bit 16 (`HSEON`). Wait for bit 17 (`HSERDY`).
- Configure PLL: `RCC_PLLCFGR` — `PLLM=4`, `PLLN=180`, `PLLP=2`, source=HSE.
- Enable PLL: `RCC_CR` bit 24 (`PLLON`). Wait for bit 25 (`PLLRDY`).
- Flash latency: `FLASH_ACR LATENCY[3:0] = 0101` (5 wait states).
- AHB=/1, APB1=/4, APB2=/2 — set in `RCC_CFGR`.
- Switch clock: `SW[1:0]=10` in `RCC_CFGR`. Wait for `SWS[1:0]=10`.

A3.2 Step A3.2 — GPIO Configuration

- Enable GPIO clocks: `RCC_AHB1ENR` bits 0 (GPIOA), 1 (GPIOB), 2 (GPIOC).
- PA2, PA3 — AF7 (USART2 TX/RX).
- PC1 (MOSI) — AF7; PC2 (MISO) — AF5; PC7 (SCK) — AF5.
- PB9 (CS) — General-purpose output, push-pull, initially HIGH.
- Set `OSPEEDR = 11` (high speed) for all SPI and UART pins.

A3.3 Step A3.3 — USART2 Configuration (115200 baud)

- Enable USART2 clock: `RCC_APB1ENR` bit 17.
- BRR: `DIV_Mantissa=24`, `DIV_Fraction=7` (APB1 = 45 MHz).
- `USART_CR1`: enable USART (bit 13), TX (bit 3), RX (bit 2), RXNEIE (bit 5).

A3.4 Step A3.4 — TIM6 (1-Second Interrupt)

- Enable TIM6: `RCC_APB1ENR` bit 4.
- `PSC = 9000` → tick = 0.1 ms (PCLK1 = 90 MHz).
- `ARR = 10000` → interrupt every 1 second.
- `TIM6_DIER` bit 0 (UIE) = 1. `TIM6_CR1` bit 0 (CEN) = 1.
- ISR: clear UIF, call `BME280_ReadAll()`, call `UART_SendSensorData()`.

A3.5 Step A3.5 — SPI2 Configuration

Enable SPI2 clock: `RCC_APB1ENR` bit 14.

```
/* SPI_CR1 configuration */
SPI2->CR1  =  (1<<2)     // MSTR      = 1   : master mode
            | (2<<3)     // BR[2:0]   = 010 : fPCLK/8
            | (0<<1)     // CPOL      = 0
            | (0<<0)     // CPHA      = 0  -> Mode 00
            | (0<<7)     // LSBFIRST  = 0   : MSB first
            | (1<<9)     // SSM       = 1   : software NSS
            | (1<<8)     // SSI       = 1
            | (0<<11);   // DFF       = 0   : 8-bit frame
SPI2->CR1 |= (1<<6);     // SPE       = 1   : enable SPI
```

A3.6 Step A3.6 — BME280 Initialisation (SPI)

- Write `0xB6` to register `0xE0` (soft reset). Delay ≥ 2 ms.
- Read register `0xD0` (chip ID). Assert value `== 0x60`.
- Read calibration registers `0x88`–`0x9F`, `0xA1`, `0xE1`–`0xE7` into typed variables.
- Write `0x01` to `0xF2` (`ctrl_hum`): `osrs_h = x1`.
- Write `0x57` to `0xF4` (`ctrl_meas`): `osrs_t=x2`, `osrs_p=x16`, mode=normal.
- Write `0x10` to `0xF5` (`config`): filter=16, `t_sb`=0.5 ms.

A3.7 Step A3.7 — Data Read and Compensation

- Burst-read 8 bytes from `0xF7` (`press_msb` ... `hum_lsb`).
- `adc_P = (raw[0]<<12)|(raw[1]<<4)|(raw[2]>>4)`
- `adc_T = (raw[3]<<12)|(raw[4]<<4)|(raw[5]>>4)`
- `adc_H = (raw[6]<<8)|raw[7]`
- Apply BME280 datasheet integer compensation formulas (Section 4.2.3).
- Convert: `comp_T/100.0` → °C; `comp_P/256/100.0` → hPa; `comp_H/1024.0` → %RH.

A4. Bare-Metal Code Snippets (SPI)

A4.1 SPI TxRx

```
uint8_t SPI_TxRx(uint8_t data) {
    while (!(SPI2->SR & (1<<1)));    // Wait TXE
    SPI2->DR = data;
    while (!(SPI2->SR & (1<<0)));    // Wait RXNE
    return (uint8_t)SPI2->DR;
}
```

A4.2 Burst Read

```
void BME280_SPI_ReadRegs(uint8_t reg, uint8_t *buf, uint8_t len)
   {
    GPIOB->ODR &= ~(1<<9);           // CS LOW
    SPI_TxRx(reg | 0x80);             // read command (MSB = 1)
    for (uint8_t i = 0; i < len; i++)
        buf[i] = SPI_TxRx(0xFF);      // dummy byte → capture MISO
    GPIOB->ODR |=  (1<<9);           // CS HIGH
}
```

A4.3 Register Write

```
void BME280_SPI_WriteReg(uint8_t reg, uint8_t data) {
    GPIOB->ODR &= ~(1<<9);
    SPI_TxRx(reg & 0x7F);             // write command (MSB = 0)
    SPI_TxRx(data);
    GPIOB->ODR |=  (1<<9);
}
```

A4.4 UART Output

```
void UART_SendString(const char *s) {
    while (*s) {
        while (!(USART2->SR & (1<<7)));
        USART2->DR = (uint8_t)(*s++);
    }
}

// In TIM6 ISR after compensation:
char msg[128];
sprintf(msg, "[SPI] Temp:%.2fC/%.2fF Pres:%.2fhPa Hum:%.2f%%\r\n"
   ,
        temp_C, temp_F, pres_hPa, hum_RH);
UART_SendString(msg);
```

A5. HAL / CubeIDE Implementation (SPI)

A5.1 CubeIDE Project Setup

1. New STM32 Project → STM32F446RETx (Nucleo-64).
2. Pinout: SPI2 Full-Duplex Master, Hardware NSS = Disabled.
3. SPI2 pins: PC1 MOSI (AF5), PC2 MISO (AF5), PC7 SCK (AF5). PB9 = GPIO_Output (CS).
4. USART2: Asynchronous, 115200, 8N1.
5. TIM6: PSC = 9000−1, ARR = 10000−1, NVIC TIM6 global interrupt enabled.
6. Clock: SYSCLK = 180 MHz via HSE PLL.
7. **Project → Generate Code.**

A5.2 HAL SPI Wrapper

```
extern SPI_HandleTypeDef hspi2;

void BME280_HAL_SPI_ReadRegs(uint8_t reg, uint8_t *buf, uint8_t
   len) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_RESET); // CS
       LOW
    uint8_t cmd = reg | 0x80;
    HAL_SPI_Transmit(&hspi2, &cmd, 1, HAL_MAX_DELAY);
    HAL_SPI_Receive (&hspi2, buf, len, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_SET);   // CS
       HIGH
}

void BME280_HAL_SPI_WriteReg(uint8_t reg, uint8_t data) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_RESET);
    uint8_t buf[2] = { reg & 0x7F, data };
    HAL_SPI_Transmit(&hspi2, buf, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_SET);
}
```

A5.3 HAL TIM6 Callback

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM6) {
        BME280_ReadAll_SPI();   // reads, compensates, converts
        char msg[128];
        sprintf(msg, "[SPI-HAL] Temp:%.2fC Pres:%.2fhPa Hum:%.2f
           %%\r\n",
                temp_C, pres_hPa, hum_RH);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg),
           HAL_MAX_DELAY);
    }
}
```

A6. Verification Tests — Part A (SPI)

Run all five tests and record results. Each test label must appear in GTKterm output.

A6.1 Test A1 — Chip ID

```
uint8_t id;
BME280_SPI_ReadRegs(0xD0, &id, 1);
char s[64];
sprintf(s, "[A1] ChipID=0x%02X (expect 0x60)\r\n", id);
UART_SendString(s);
```

Expected	Pass Condition	On Fail
`[A1] ChipID=0x60`	`id == 0x60`	Check CS polarity, CPOL/CPHA, MOSI/MISO swap

A6.2 Test A2 — UART Loopback

```
UART_SendString("[A2] UART OK\r\n");  // call before BME280 init
```

Expected in GTKterm immediately after reset: `[A2] UART OK`

A6.3 Test A3 — TIM6 Heartbeat

```
static uint32_t ticks = 0;
// In TIM6 ISR: ticks++;
// sprintf(msg, "[A3] Tick:%lu\r\n", ticks); UART_SendString(msg)
   ;
```

Expected: counter increments exactly once per second.

A6.4 Test A4 — Sensor Plausibility

```
if      (temp_C  < 15.0f || temp_C  > 40.0f) UART_SendString("[A4
   ] Temp FAIL\r\n");
else if (pres_hPa < 900   || pres_hPa > 1100) UART_SendString("[
   A4] Pres FAIL\r\n");
else if (hum_RH  < 0      || hum_RH  > 100  ) UART_SendString("[
   A4] Hum  FAIL\r\n");
else                                          UART_SendString("[
   A4] Plausibility PASS\r\n");
```

A6.5 Test A5 — HAL vs Bare-Metal Comparison

Flash bare-metal, record three readings. Flash HAL version, record three. Fill in the table.

Reading #	BM Temp (°C)	HAL Temp (°C)	Diff	Pass (≤0.1 °C)?
1	_____	_____	_____	
2	_____	_____	_____	
3	_____	_____	_____	

A7. Expected UART Output — Part A

```
========================================
  BME280 via SPI  --  CSE 2206 Lab A
========================================
Temperature :  28.45 C  /  83.21 F
Pressure    : 1012.87 hPa
Humidity    :  61.32 %RH
----------------------------------------
```

If all values read `0.00`: check CS pin (PB9) and SPI Mode (CPOL=0, CPHA=0). If chip ID = `0xFF`: sensor not responding — verify MOSI/MISO wiring.

A8. Part A Evaluation Criteria

Criterion	Description	CLO	Marks
Clock & GPIO	180 MHz PLL, correct AFs for SPI+UART pins	CLO 1	5
SPI2 Configuration	Master mode, Mode 00, correct baud, 8-bit, bare-metal	CLO 1	10
BME280 SPI Init	Reset, chip ID, calibration read, sensor config	CLO 3	8
Data Read & Compensation	Burst read, correct ADC reconstruction, formulas	CLO 3	8
Unit Conversion & UART	Correct °C→°F, Pa→hPa, clean GTKterm output	CLO 5	4
HAL / CubeIDE SPI	Working HAL implementation, output matches bare-metal	CLO 4	10
Verification Tests A1–A5	All tests implemented and results recorded	CLO 1,3,4,5	5
Part A Total			**50**

PART B
I²C Communication with BME280

2-Wire I²C — STM32 as Master — Bare-Metal + HAL

B1. I²C Protocol Overview

Inter-Integrated Circuit (I²C) is a synchronous, half-duplex, 2-wire serial bus. Unlike SPI, it uses only two lines (SCL and SDA) and supports multiple devices on the same bus via 7-bit or 10-bit addressing. The BME280's I²C slave address is determined by its `SDO/ADDR` pin.

Signal	Full Name	Direction	Purpose
SCL	Serial Clock Line	Master → Slave	Clock generated by master; synchronises data
SDA	Serial Data Line	Bidirectional	Both master and slave transmit on this line

B1.1 BME280 I²C Slave Address

SDO/ADDR Pin	7-bit Address	8-bit Write	8-bit Read
Connected to GND	0x76	0xEC	0xED
Connected to VCC	0x77	0xEE	0xEF

> Default (SDO to GND): address = `0x76`. Confirm with your hardware.

B1.2 I²C Transaction Structure

Write to a BME280 register:

```
START → [slave_addr | 0] → ACK → [reg_addr] → ACK → [data] → ACK
  → STOP
```

Read from a BME280 register (register-then-read):

```
START → [slave_addr | 0] → ACK → [reg_addr] → ACK →
RESTART → [slave_addr | 1] → ACK → [data byte(s)] → NACK → STOP
```

B1.3 SPI vs I²C — Key Differences

Property	SPI (Part A)	I²C (Part B)
Wires	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
Speed	Up to 45 Mbps (fPCLK/2)	Std 100 kHz / Fast 400 kHz
Multiple Slaves	One CS per slave	Address-based, shared bus
Full Duplex	Yes	No (half-duplex)
Implementation	Simpler register config	START/STOP/ACK state machine
Pin Count	Higher	Lower — ideal for dense boards

B2. Hardware Connections — I²C

I²C Signal	STM32 Pin	Alt Function	BME280 Pin
SCL (Clock)	PB6	AF4 (I²C1)	SCK
SDA (Data)	PB7	AF4 (I²C1)	SDI/SDO
UART TX	PA2	AF7 (USART2)	—
UART RX	PA3	AF7 (USART2)	—

Both SCL and SDA require external pull-up resistors (4.7 kΩ to 3.3 V). Many BME280 breakout boards include these; if using a bare chip, add them manually.

I²C does **not** use a CS pin. The sensor is selected by its 7-bit address transmitted at the start of every transaction.

B3. Bare-Metal Implementation (Register-Level)

Repeat the shared peripherals (clock, GPIO for UART, USART2, TIM6) exactly as in Part A Steps A3.1–A3.4. Only the bus-specific configuration differs.

B3.1 Step B3.1 — I²C1 GPIO Configuration

- Enable GPIOB clock: `RCC_AHB1ENR` bit 1 (already set in Part A).
- PB6 (SCL) and PB7 (SDA): set `MODER = 10` (Alternate Function).
- Set AF4 for both PB6 and PB7 in `GPIOB_AFRL`.

- Set `OTYPER = 1` (open-drain) for PB6 and PB7 — **mandatory for I²C**.
- Set `OSPEEDR = 01` (medium speed) for PB6 and PB7.
- Set `PUPDR = 01` (pull-up) for PB6 and PB7 (or rely on external resistors).

B3.2 Step B3.2 — I²C1 Peripheral Configuration

Enable I²C1 clock: `RCC_APB1ENR` bit 21. Reset I²C1: set then clear bit 21 in `RCC_APB1RSTR`.

```
/* APB1 peripheral clock = 45 MHz */

/* CR2: FREQ field = 45 (APB1 in MHz) */
I2C1->CR2 = 45;

/* CCR: Standard mode (Sm), 100 kHz
   T_high = T_low = 5 us; CCR = T_high / T_pclk1
   CCR = 5000 ns / (1000/45 ns) = 225              */
I2C1->CCR = 225;

/* TRISE: max rise time = 1000 ns in Sm
   TRISE = (1000 ns / T_pclk1) + 1 = 45 + 1 = 46  */
I2C1->TRISE = 46;

/* Enable I2C1: CR1 PE bit */
I2C1->CR1 |= (1<<0);    // PE = 1
```

B3.3 Step B3.3 — BME280 Initialisation (I²C)

- I²C write `0xB6` to register `0xE0` (soft reset). Delay ≥ 2 ms.
- I²C read register `0xD0` (chip ID). Assert value `== 0x60`.
- I²C read calibration registers `0x88`–`0x9F`, `0xA1`, `0xE1`–`0xE7`.
- Write `0x01` to `0xF2`, `0x57` to `0xF4`, `0x10` to `0xF5`.

B3.4 Step B3.4 — Data Read and Compensation (I²C)

- I²C burst-read 8 bytes from `0xF7` → same ADC reconstruction as Part A.
- Apply identical compensation formulas and unit conversions.
- Transmit via USART2 every 1 second via TIM6 ISR.

B4. Bare-Metal Code Snippets (I²C)

B4.1 I²C Write Register

```
#define BME280_I2C_ADDR  0x76

void I2C_WriteReg(uint8_t reg, uint8_t data) {
    /* Generate START */
```

```
    I2C1->CR1 |= (1<<8);                          // START
    while (!(I2C1->SR1 & (1<<0)));                 // Wait SB

    /* Send slave address + WRITE (bit0 = 0) */
    I2C1->DR = (BME280_I2C_ADDR << 1) | 0;
    while (!(I2C1->SR1 & (1<<1)));                 // Wait ADDR
    (void)I2C1->SR2;                               // Clear ADDR flag

    /* Send register address */
    I2C1->DR = reg;
    while (!(I2C1->SR1 & (1<<7)));                 // Wait TXE

    /* Send data */
    I2C1->DR = data;
    while (!(I2C1->SR1 & (1<<2)));                 // Wait BTF

    /* Generate STOP */
    I2C1->CR1 |= (1<<9);
}
```

B4.2 I²C Burst Read

```
void I2C_ReadRegs(uint8_t reg, uint8_t *buf, uint8_t len) {
    /* Phase 1: write register pointer */
    I2C1->CR1 |= (1<<8);                          // START
    while (!(I2C1->SR1 & (1<<0)));
    I2C1->DR = (BME280_I2C_ADDR << 1) | 0;        // addr + WRITE
    while (!(I2C1->SR1 & (1<<1)));
    (void)I2C1->SR2;
    I2C1->DR = reg;
    while (!(I2C1->SR1 & (1<<2)));                 // Wait BTF

    /* Phase 2: repeated START + READ */
    I2C1->CR1 |= (1<<10) | (1<<8);                // ACK=1, RESTART
    while (!(I2C1->SR1 & (1<<0)));
    I2C1->DR = (BME280_I2C_ADDR << 1) | 1;        // addr + READ
    while (!(I2C1->SR1 & (1<<1)));
    (void)I2C1->SR2;

    for (uint8_t i = 0; i < len; i++) {
        if (i == len - 1) I2C1->CR1 &= ~(1<<10); // NACK on last
           byte
        while (!(I2C1->SR1 & (1<<6)));                 // Wait RXNE
        buf[i] = I2C1->DR;
    }
    I2C1->CR1 |= (1<<9);                          // STOP
}
```

B4.3 UART Output (same function as Part A)

```
// Reuse UART_SendString() from Part A
char msg[128];
sprintf(msg, "[I2C] Temp:%.2fC/%.2fF Pres:%.2fhPa Hum:%.2f%%\r\n"
    ,
        temp_C, temp_F, pres_hPa, hum_RH);
UART_SendString(msg);
```

B5. HAL / CubeIDE Implementation (I²C)

B5.1 CubeIDE Project Setup

1. New STM32 Project → STM32F446RETx.
2. Pinout: I2C1 → Mode = I2C. Pins: PB6 (SCL), PB7 (SDA).
3. I2C1 Parameters: Speed = Standard Mode (100 kHz), own address = irrelevant (master).
4. USART2 Asynchronous 115200 8N1. TIM6 as in Part A.
5. Clock: SYSCLK = 180 MHz via HSE PLL. **Generate Code.**

B5.2 HAL I²C Wrapper Functions

```
extern I2C_HandleTypeDef hi2c1;
#define BME280_ADDR  (0x76 << 1)   // HAL uses 8-bit address

void BME280_HAL_I2C_WriteReg(uint8_t reg, uint8_t data) {
    uint8_t buf[2] = { reg, data };
    HAL_I2C_Master_Transmit(&hi2c1, BME280_ADDR, buf, 2,
       HAL_MAX_DELAY);
}

void BME280_HAL_I2C_ReadRegs(uint8_t reg, uint8_t *buf, uint8_t
   len) {
    HAL_I2C_Master_Transmit(&hi2c1, BME280_ADDR, &reg, 1,
       HAL_MAX_DELAY);
    HAL_I2C_Master_Receive (&hi2c1, BME280_ADDR, buf, len,
       HAL_MAX_DELAY);
}

/* Cleaner alternative using the Mem API: */
void BME280_HAL_I2C_MemRead(uint8_t reg, uint8_t *buf, uint8_t
   len) {
    HAL_I2C_Mem_Read(&hi2c1, BME280_ADDR, reg,
                     I2C_MEMADD_SIZE_8BIT, buf, len,
                        HAL_MAX_DELAY);
}
```

B5.3 HAL TIM6 Callback (I²C)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM6) {
        BME280_ReadAll_I2C();
        char msg[128];
        sprintf(msg, "[I2C-HAL] Temp:%.2fC Pres:%.2fhPa Hum:%.2f
           %%\r\n",
                temp_C, pres_hPa, hum_RH);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg),
           HAL_MAX_DELAY);
    }
}
```

B6. Verification Tests — Part B (I²C)

Run all five tests and record results in GTKterm.

B6.1 Test B1 — Chip ID over I²C

```
uint8_t id;
I2C_ReadRegs(0xD0, &id, 1);
char s[64];
sprintf(s, "[B1] ChipID=0x%02X (expect 0x60)\r\n", id);
UART_SendString(s);
```

Expected	Pass Condition	On Fail
`[B1] ChipID=0x60`	`id == 0x60`	Check SDA/SCL pull-ups, I²C address (0x76 vs 0x77), OTYPER = open-drain

B6.2 Test B2 — I²C ACK Check

```
/* After START, send slave address and verify ADDR flag is set (
   ACK received) */
I2C1->CR1 |= (1<<8);                    // START
while (!(I2C1->SR1 & (1<<0)));
I2C1->DR = (BME280_I2C_ADDR << 1) | 0;
uint32_t timeout = 100000;
while (!(I2C1->SR1 & (1<<1)) && --timeout);
if (!timeout) UART_SendString("[B2] I2C ACK FAIL\r\n");
else { (void)I2C1->SR2; UART_SendString("[B2] I2C ACK OK\r\n"); }
I2C1->CR1 |= (1<<9);                    // STOP
```

B6.3 Test B3 — TIM6 Heartbeat

Reuse the heartbeat from Part A. Label output `[B3]`. Expected: counter increments exactly once per second.

B6.4 Test B4 — Sensor Plausibility

```
if (temp_C<15||temp_C>40||pres_hPa<900||pres_hPa>1100||hum_RH<0||
   hum_RH>100)
    UART_SendString("[B4] Plausibility FAIL\r\n");
else
    UART_SendString("[B4] Plausibility PASS\r\n");
```

B6.5 Test B5 — HAL vs Bare-Metal (I²C)

Reading #	BM Temp (°C)	HAL Temp (°C)	Diff	Pass (≤0.1 °C)?
1	_____	_____	_____	
2	_____	_____	_____	
3	_____	_____	_____	

B7. Expected UART Output — Part B

```
========================================
  BME280 via I2C  --  CSE 2206 Lab B
========================================
Temperature :  28.46 C  /  83.23 F
Pressure    : 1012.85 hPa
Humidity    :  61.35 %RH
----------------------------------------
```

If readings are stuck at 0 or show garbage values: check pull-up resistors on SDA/SCL. Missing pull-ups are the most common I²C fault.

B8. Part B Evaluation Criteria

Criterion	Description	CLO	Marks
I²C GPIO Setup	Open-drain, AF4, pull-ups for PB6/PB7	CLO 2	5
I²C1 Configuration	CR2 FREQ, CCR, TRISE correct for 100 kHz, bare-metal	CLO 2	10
BME280 I²C Init	Reset, ACK check, chip ID, calibration, sensor config	CLO 3	8
Data Read & Compensation	Burst read via I²C, correct reconstruction, formulas	CLO 3	8
Unit Conversion & UART	Correct °C→°F, Pa→hPa, clean GTKterm output	CLO 5	4
HAL / CubeIDE I²C	Working HAL `Mem_Read` implementation, output matches	CLO 4	10
Verification Tests B1–B5	All tests implemented and results recorded	CLO 2,3,4,5	5
Part B Total			**50**

5. Final Comparison: SPI vs I²C

After completing both parts, fill in this summary table using your own measurements and observations.

Aspect	Part A — SPI	Part B — I²C
Wires to BME280	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
STM32 Peripheral	SPI2	I²C1
Bare-Metal Config	∼6 register writes	∼5 register writes + open-drain
HAL API	`HAL_SPI_Transmit/Receive`	`HAL_I2C_Mem_Read/Write`
Max Bus Speed	45 Mbps (fPCLK/2)	400 kHz (Fast mode)
Pull-up Resistors	Not required	Required (4.7 kΩ to 3.3 V)
CS Pin Required	Yes (PB9)	No — address-based
Multiple Slaves	One CS per slave	Address-based sharing
Your Temp (°C)	_____	_____
Your Pressure (hPa)	_____	_____
Readings Match?	—	Yes / No (within 0.1 °C)

Discussion Questions (answer in your lab report):

1. When would you choose SPI over I²C in a real product? Consider pin count, speed requirements, number of peripherals, and board complexity.
2. The BME280 outputs identical sensor data regardless of whether SPI or I²C is used. Why, then, is the firmware compensation code the same in both parts?

6. Learning Outcomes

Upon successful completion of both parts, students will have demonstrated:

- A working understanding of both SPI and I²C bus protocols at the signal level.
- The ability to configure STM32 SPI2 and I²C1 peripherals via direct register access.
- Competence in reading sensor datasheets to configure operating modes and parse raw data registers.
- Implementation of fixed-point compensation algorithms from manufacturer specifications.
- Proficiency in GTKterm for real-time UART verification and debugging.
- The ability to replicate bare-metal designs using STM32 HAL and critically compare both.
- Systematic hardware debugging skills through structured verification tests.
- The ability to evaluate trade-offs between two industry-standard communication protocols.

> These skills directly underpin professional embedded firmware development in IoT, automotive electronics, industrial automation, and medical device engineering.

— End of Assignment 3 —